COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE-010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07/30/2026 | **DATE SUBMITTED:** | 07/30/2026 |
| **NAME:** | Julian Benedict D.R. Barizo | **INSTRUCTOR:** | Jimlord Quejado |

**ACTIVITY NO. 3.1 [Linked Lists]**

**1. PROCEDURE OUTPUT**

| | |
|---|---|
| **Screenshot** |  |
| **Discussion** | Initialized three nodes consisting of characters and putting them on their respective linked list. |

```
#include <iostream>
#include "singly_ll.h"
#include "doubly_ll.h"

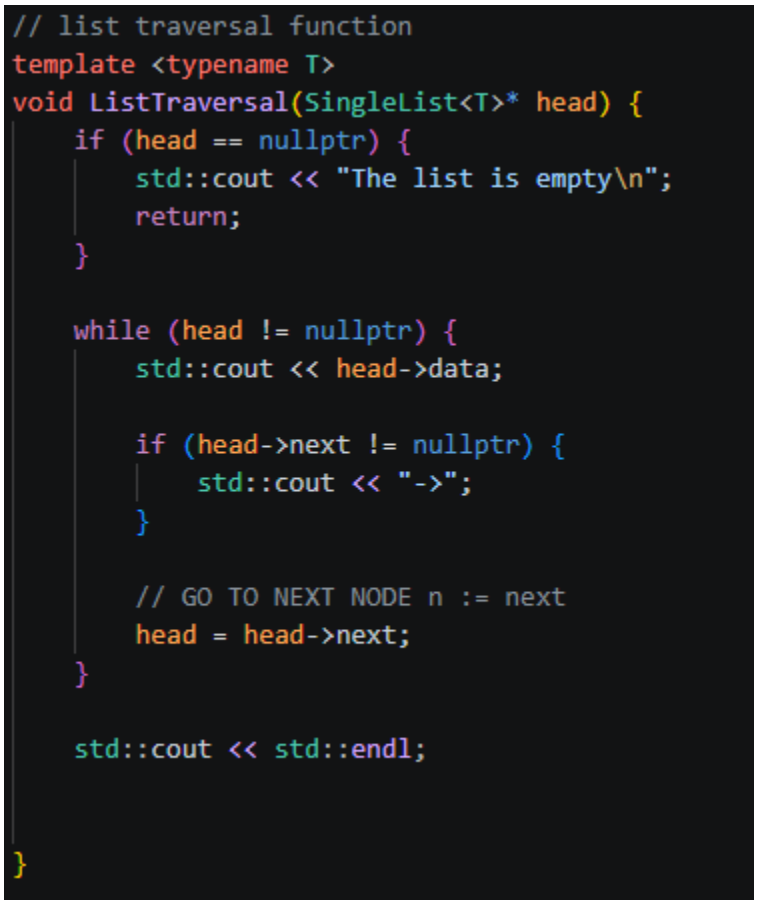
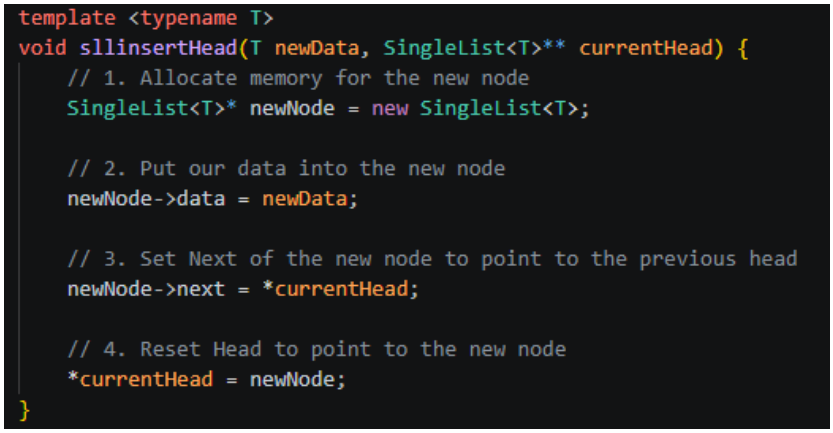
int main () {
    // create a list "C P E"

    DoubleList<char>* head = CreateNewNode('C');
    DoubleList<char>* second = CreateNewNode('P');
    DoubleList<char>* third = CreateNewNode('E');

    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;
```

Table 3-1. Output of Initial/Simple Implementation

| Operation | Screenshot |
|---|---|
| Traversal |  |
| Insertion at head |  |

```cpp
// list traversal function
template <typename T>
void ListTraversal(SingleList<T>* head) {
    if (head == nullptr) {
        std::cout << "The list is empty\n";
        return;
    }

    while (head != nullptr) {
        std::cout << head->data;

        if (head->next != nullptr) {
            std::cout << "->";
        }

        // GO TO NEXT NODE n := next
        head = head->next;
    }

    std::cout << std::endl;


}
```

```cpp
template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead) {
    // 1. Allocate memory for the new node
    SingleList<T>* newNode = new SingleList<T>;

    // 2. Put our data into the new node
    newNode->data = newData;

    // 3. Set Next of the new node to point to the previous head
    newNode->next = *currentHead;

    // 4. Reset Head to point to the new node
    *currentHead = newNode;
}
```

Insertion at any part of the list

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode) {
    // 1. Check if it is the head node (previous node is null)
    if (prevNode == nullptr) {
        // 2. If null, print "Previous value cannot be null."
        std::cout << "Previous value cannot be null" << std::endl;
        return;
    }

    // Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 4.
    newNode->data = newData;

    // 5.
    newNode->next = prevNode->next;

    // 6.
    prevNode->next = newNode;
}
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head) {
    // 1. Allocate a new node
    SingleList<T>* newNode = new SingleList<T>;

    // 2. Dereference to the head node
    SingleList<T>* currentNode = *head;

    // 3. Store data in new node
    newNode->data = newData;

    // 4. Point to Null
    newNode->next = nullptr;

    // 5. Traverse the list until it is pointed to null
    while (currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

| | |
|---|---|
| Deletion of a node |  |

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head) {
    // if there is nothing to delete
    if (*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

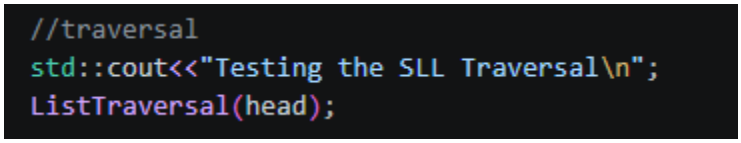
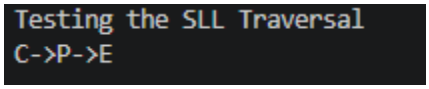
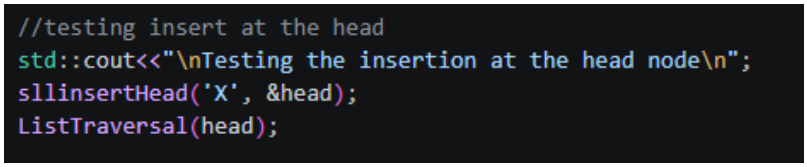
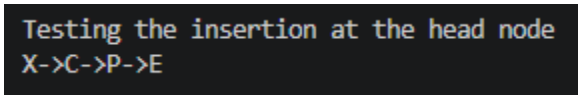
    // searching the node; 1. the node doesn't exist + we found the data
    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    // if data not found CurrNode == nullptr in the while loop:
    if (currNode == nullptr) {
        std::cout << "the data is not found \n" << std::endl;
        return;
    }

    // if data was found:
    if (prevNode == nullptr) {
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2. Code for the List Operations

| | | |
|---|---|---|
| a. | **Source Code** |  |
| | **Console** |  |
| b. | **Source Code** |  |
| | **Console** |  |

```
//traversal
std::cout<<"Testing the SLL Traversal\n";
ListTraversal(head);
```

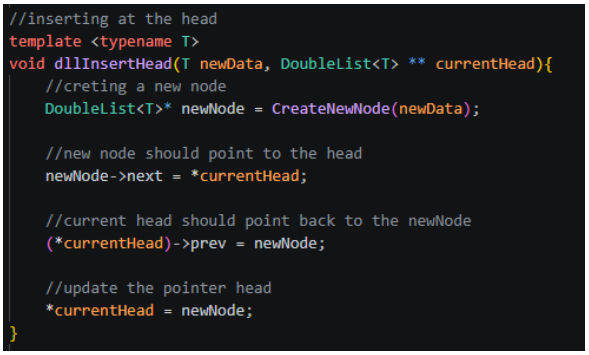
```
Testing the SLL Traversal
C->P->E
```

```
//testing insert at the head
std::cout<<"\nTesting the insertion at the head node\n";
sllinsertHead('X', &head);
ListTraversal(head);
```

```
Testing the insertion at the head node
X->C->P->E
```

| | | |
|---|---|---|
| c. | **Source Code** | `//general insert`<br>`std::cout<<"\nTesting the General Insertion\n";`<br>`sllGeneralInsert('L', second);`<br>`ListTraversal(head);` |
| | **Console** | `Testing the General Insertion`<br>`X->C->P->L->E` |
| d. | **Source Code** | `//testing the insertion at the end`<br>`std::cout<<"\nTesting the insertion at the end\n";`<br>`sllinsertEnd('0', &head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing the insertion at the end`<br>`X->C->P->L->E->0` |
| e. | **Source Code** | `//testing the deletion of a node`<br>`std::cout<<"\nTesting the deletion of a node\n";`<br>`sllDeleteNode('L', &head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing the deletion of a node`<br>`X->C->P->E->0` |
| f. | **Source Code** | `//testing the deletion of the whole list`<br>`std::cout<<"\nTesting the deletion of the whole list\n";`<br>`sllDeleteList(&head);`<br>`ListTraversal(head);` |
| | **Console** | `Testing the deletion of the whole list`<br>`The list is empty` |

Table 3-3. Code and Analysis for Singly Linked Lists

| Screenshot(s) | Analysis |
| --- | --- |
|  <br>```<br>template <typename T><br>void dllTraverse(DoubleList<T>* currentNode){<br><br>    DoubleList<T> *tail;<br><br>    //check if the lis is empty<br>    if(currentNode == nullptr){<br>        std::cout<<"the list is empty";<br>        return;<br>    }<br>    std::cout<<"Forwarding: \n";<br>    //move forward<br>    while(currentNode != nullptr){<br>        std::cout<< currentNode->data << " ";<br>        tail = currentNode;<br>        currentNode = currentNode->next;<br>    }<br>    //add a next line<br>    std::cout<<std::endl;<br>    //currentNode = currentNode->prev;<br>    std::cout<<"Backward: \n";<br>    //move backward<br>    while(tail != nullptr){<br>        std::cout<< tail->data<<" ";<br>        tail = tail->prev;<br>    }<br>}<br>``` | This function walks the list forward from currentNode to nullptr, printing each data value while saving the last node visited in tail; once it hits the end, it uses that saved tail pointer to walk prev links all the way back to the head, printing the list again in reverse. |
|  <br>```<br>//inserting at the head<br>template <typename T><br>void dllInsertHead(T newData, DoubleList<T> ** currentHead){<br>    //creting a new node<br>    DoubleList<T>* newNode = CreateNewNode(newData);<br><br>    //new node should point to the head<br>    newNode->next = *currentHead;<br><br>    //current head should point back to the newNode<br>    (*currentHead)->prev = newNode;<br><br>    //update the pointer head<br>    *currentHead = newNode;<br>}<br>``` | This function creates a new node holding newData, then points its next to whatever the current head is, and points the old head's prev back to the new node linking the new node into the list on both sides before it becomes the head. |

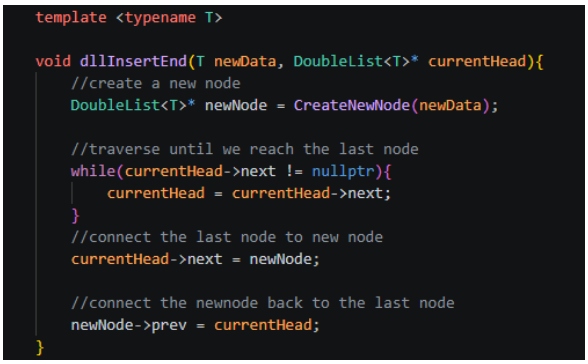
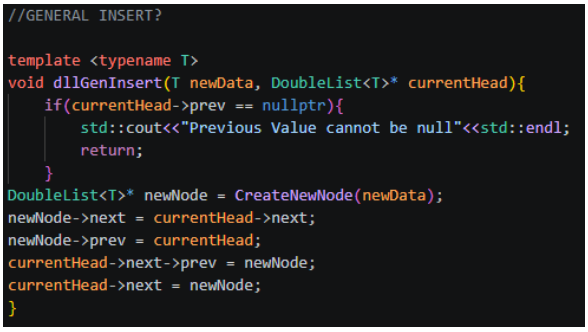
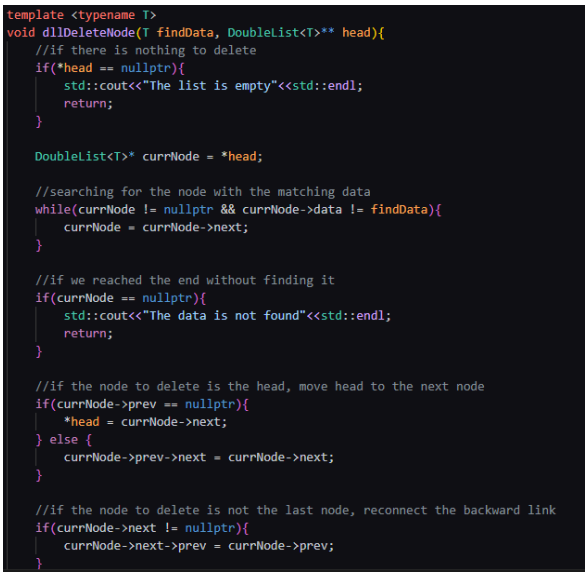
| | |
|---|---|
|  | This function creates a new node, then walks currentHead forward using ->next until it lands on the last node in the list (the one whose next is nullptr), rather than starting from a fresh copy of the head. It then links the new node in on both sides — setting the old last node's next to point forward to it, and the new node's prev to point backward to the old last node so the list stays correctly linked in both directions. |
|  | This function inserts a new node immediately after currentHead, but first guards against currentHead->prev == nullptr — meaning it refuses to run if given the head node, since general insertion is meant for the middle of the list, not the ends. It then relinks four pointers in sequence so the new node sits correctly between currentHead and whatever came after it. |
|  | This function first searches for a node matching findData by walking next pointers from head, printing a "not found" message and returning early if it reaches nullptr without a match. Once found, it handles two cases moving *head forward if the target was the head itself, and relinking the neighboring nodes' prev/next pointers before freeing the deleted node with delete, |

Table 3-4. Modified Operations for Doubly Linked List

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <string>
#include "circular_ll.h"

int main() {
    // 1. Create a playlist using multiple songs
    CircularList<std::string>* playlist =
cdllCreatePlaylist<std::string>(
        {"Bohemian Rhapsody", "Hotel California", "Imagine"});

    std::cout << "--- Initial Playlist ---\n";
    cdllPlayAll(playlist);

    // 2. Add a song to the playlist
    std::cout << "\n--- Adding 'Wonderwall' ---\n";
    cdllAddSong<std::string>("Wonderwall", &playlist);
    cdllPlayAll(playlist);

    // 3. Remove a song from the playlist
    std::cout << "\n--- Removing 'Hotel California' ---\n";
    cdllDeleteSong<std::string>("Hotel California", &playlist);
    cdllPlayAll(playlist);

    // 4. Move through the playlist with next/previous
    std::cout << "\n--- Testing Next / Previous ---\n";
    CircularList<std::string>* current = playlist;
    current = cdllNext(current);       // move forward
    current = cdllNext(current);       // move forward again
    current = cdllPrevious(current);   // move back one

    // 5. Play all songs one more time to confirm the list is intact
    std::cout << "\n--- Final Playlist ---\n";
    cdllPlayAll(playlist);

    // 6. Clean up
    cdllDeletePlaylist(&playlist);
```

```
    return 0;
}
```

```
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>

// node creation - a circular doubly linked list node
// each node stores one song and links both forward (next) and backward
(prev)
template <typename T>
class CircularList {
public:
    T data; // this contains the song data to be stored in the node
    CircularList<T>* next = nullptr;
    CircularList<T>* prev = nullptr;
};

// creates a single node; a lone node points to itself in both
directions
// so the list is always circular, even with just one song
template <typename T>
CircularList<T>* CreateCircularNode(T newData) {
    CircularList<T>* newNode = new CircularList<T>;
    newNode->data = newData;
    newNode->next = newNode;
    newNode->prev = newNode;
    return newNode;
}

// ADD A SONG (insert at the end, i.e. right before head)
template <typename T>
void cdllAddSong(T newData, CircularList<T>** head) {
    CircularList<T>* newNode = CreateCircularNode(newData);

    // 1. if the playlist is empty, the new song becomes the head
    if (*head == nullptr) {
```

```
        *head = newNode;
        return;
    }

    // 2. find the tail (the node right before head)
    CircularList<T>* tail = (*head)->prev;

    // 3. splice the new node in between tail and head
    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *head;
    (*head)->prev = newNode;
}

// CREATE A PLAYLIST from multiple songs at once
template <typename T>
CircularList<T>* cdllCreatePlaylist(std::initializer_list<T> songs) {
    CircularList<T>* head = nullptr;

    for (const T& song : songs) {
        cdllAddSong(song, &head);
    }

    return head;
}

// REMOVE A SONG from the playlist
template <typename T>
void cdllDeleteSong(T findData, CircularList<T>** head) {
    // 1. nothing to delete
    if (*head == nullptr) {
        std::cout << "The playlist is empty\n";
        return;
    }

    CircularList<T>* current = *head;

    // 2. search the circle for the song (do-while since it's circular,
```

```
    //    a plain while would never start)
    do {
        if (current->data == findData) {
            break;
        }
        current = current->next;
    } while (current != *head);

    // 3. song not found -> we looped all the way back without a match
    if (current->data != findData) {
        std::cout << "The song was not found\n";
        return;
    }

    // 4. only one song left in the playlist
    if (current->next == current) {
        *head = nullptr;
    } else {
        current->prev->next = current->next;
        current->next->prev = current->prev;

        // if we deleted the head, move head to the next song
        if (current == *head) {
            *head = current->next;
        }
    }

    delete current;
}

// PLAY ALL SONGS (traverse once around the circle starting at head)
template <typename T>
void cdllPlayAll(CircularList<T>* head) {
    if (head == nullptr) {
        std::cout << "The playlist is empty\n";
        return;
    }
```

```
    CircularList<T>* current = head;

    std::cout << "Now playing: \n";
    do {
        std::cout << current->data;

        if (current->next != head) {
            std::cout << " -> ";
        }

        current = current->next;
    } while (current != head);

    std::cout << std::endl;
}

// NEXT SONG - move forward one song from the current position
template <typename T>
CircularList<T>* cdllNext(CircularList<T>* current) {
    if (current == nullptr) {
        std::cout << "The playlist is empty\n";
        return nullptr;
    }

    current = current->next;
    std::cout << "Next song: " << current->data << std::endl;
    return current;
}

// PREVIOUS SONG - move backward one song from the current position
template <typename T>
CircularList<T>* cdllPrevious(CircularList<T>* current) {
    if (current == nullptr) {
        std::cout << "The playlist is empty\n";
        return nullptr;
    }

    current = current->prev;
```

```
    std::cout << "Previous song: " << current->data << std::endl;
    return current;
}

// DELETE THE WHOLE PLAYLIST
template <typename T>
void cdllDeletePlaylist(CircularList<T>** head) {
    if (*head == nullptr) return;

    CircularList<T>* current = (*head)->next;

    while (current != *head) {
        CircularList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    delete *head;
    *head = nullptr;
}

#endif
```

```
--- Initial Playlist ---
Now playing:
Bohemian Rhapsody -> Hotel California -> Imagine

--- Adding 'Wonderwall' ---
Now playing:
Bohemian Rhapsody -> Hotel California -> Imagine -> Wonderwall

--- Removing 'Hotel California' ---
Now playing:
Bohemian Rhapsody -> Imagine -> Wonderwall

--- Testing Next / Previous ---
Next song: Imagine
Next song: Wonderwall
Previous song: Imagine

--- Final Playlist ---
Now playing:
Bohemian Rhapsody -> Imagine -> Wonderwall
```

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Formatting, Code assistance, explanation of concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **Usage of AI was for code assistance explaining how to do circular linked lists and properly typing the correct syntaxes. Explained where to also point the varying pointers and why it should point there.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.